Summary

133 distinct endpoints across four APIs, with 131 covered. Yoco was run twice to compare input formats, producing 753 tests across 5 input runs. This run demonstrated four input formats: OpenAPI, Swagger, Postman, and HTML documentation. The same approach extends to other documentation formats and source types.

Results at a Glance

Target	Input Source	Endpoints Covered	Response Scenarios Covered	Tests
ABSA	OpenAPI 3.0.1	5/5 (100%)	30/40 (75%)	49
African Bank	Swagger 2.0	39/39 (100%)	170 declared, 161 tested, 9 excluded	195
Peach Payments	Postman Collection	33/35 (94.3%)	33/35 (94.3%)	66
Yoco	OpenAPI 3.1	42/42 (100%)	205/206 (99.5%)	250
Yoco	HTML Documentation	54/54 (100%)	180/230 (78.3%)	193

What the Tool Observed

African Bank (Grindrod) - Swagger 2.0

The tool read the published Grindrod Bank Swagger 2.0 specification and identified that one response template has been applied uniformly across all POST operations: every one of the 13 POST endpoints declares an identical **200, 201, 401, 403, 404** response block, regardless of what each operation does. All 23 GET endpoints carry the same **200, 401, 403, 404** pattern.

The consequence is that 201 Created appears on two read-only validation endpoints (**/validate/accountnumber** and **/verifications/account**) that do not create resources, and 201 Created appears on a PUT to an existing resource (**/fica/individuals/{ficaId}**). The spec also declares both 200 OK with a documented response body and 204 No Content on the same PATCH operation - two mutually exclusive outcomes for one operation. Finally, 404 Not Found is declared on two root collection POST operations (**/customers** and **/users**) whose paths carry no resource identifier that could be missing.

170 response scenarios declared. 161 tested. 9 excluded.

Peach Payments - Postman Collection

The tool parsed not just the request list but the collection's test scripts. It identified that two of the 35 routes - **{{batchUrl}}** and **{{paymentRedirectUrl}}** - are server-generated URLs set at runtime by a preceding request's test script via **pm.collectionVariables.set(...)**. Neither has a fixed path anywhere in the collection; both are populated dynamically from the response of a prior request. They cannot be represented as static routes in a coverage matrix.

33 of 35 endpoints tested. 2 excluded - dynamic-target URLs with no fixed path in the source documentation.

What Each Suite Includes

- Self-contained test project (configure environment variables and run)
- Human-readable test names describing each scenario (Given/When/Then format)
- Automatic verification that every successful response matches the documented structure
- Tests for unauthorised access (401, 403)

- Tests for invalid input (400)
- Tests that can create their own data; payloads may need to conform to your environment's validation rules
- Setup instructions and documented environment variables

Input Format Matters

The same API (Yoco) was run through two different input formats to demonstrate the difference:

	OpenAPI Spec	HTML Docs
Endpoints discovered	42	54
Response scenarios to test	206	230
Tests generated	250	193

This comparison shows that output depth depends on the completeness of the source material provided. In this Yoco sample, the HTML documentation covered a broader surface area than the supplied OpenAPI file, so it yielded more endpoints and response scenarios. Bottom line: both formats produce strong, usable output. The operational advantage of OpenAPI is its consistency, structure, and suitability for automation; raw HTML documentation remains a viable input when that is the source material available.

Delivery Contents

Per target:

- Zip file with the complete test suite (source only, run npm install to set up)
- README with setup instructions
- .env.example with all required and optional variables documented
- Coverage report showing the endpoint and response code matrix

Assessment

Ready to share. Four of five input runs achieved 100% endpoint coverage. The fifth (Peach Payments) reached 33 of 35, with the remaining 2 endpoints excluded as dynamic-target URLs that cannot be represented as static routes. All four input formats were demonstrated, and every suite is a standalone project that can be configured and executed against a target environment. Tests are human-readable and structured for maintainability. The snapshot represents what the solution produces from documentation alone.

These deliverables are documentation-derived and static by design. Response structures have not been validated against a live target environment. Live execution is a separate step used to validate behaviour and refine the suite against runtime conditions.